Cafe Data Sales Analysis Report

Project Overview

This project explores a dataset of 10,000 cafe transactions from 2023 to learn how customers buy, how well sales perform, and how they prefer to pay. This information helps businesses make smart decisions and improve their sales.

Business Objective

The main objective of this project is;

- To analyze historical sales data and identify overall sales trends and patterns.
- To determine top-performing and underperforming products based on revenue and sales volume.
- To identify peak sales periods by month and week.
- To provide data-driven recommendations to reduce waste and improve profitability.

Dataset Description

The selected dataset was the Dirty Cafe Sales Dataset 2023, uploaded by Ahmed Mohamed to Kaggle. This dataset contains 10000 records and 8 attributes, including:

Attributes Name	Description
Transaction ID	Unique identifier of transaction records
Item	Name of item purchased
Quantity	Number of units purchased
Price Per Unit	The price of a single unit of the item
Total Spent	Total amount spent on the transaction
Payment Method	Method of payment used (Digital Wallet, Cash, or Credit Card)
Location	Where the order was consumed
Transaction Date	The date of the transaction

[10]:
```
# Check data size
df.shape
```

[10]:
```
(10000, 8)
```

[11]:
```
# show first 5 rows data
df.head()
```

[11]:

	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11

[12]:
```
# show last 5 rows data
df.tail()
```

[12]:

	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
9995	TXN_7672686	Coffee	2	2.0	4.0	NaN	UNKNOWN	2023-08-30
9996	TXN_9659401	NaN	3	NaN	3.0	Digital Wallet	NaN	2023-06-02
9997	TXN_5255387	Coffee	4	2.0	8.0	Digital Wallet	NaN	2023-03-02
9998	TXN_7695629	Cookie	3	NaN	3.0	Digital Wallet	NaN	2023-12-02
9999	TXN_6170729	Sandwich	3	4.0	12.0	Cash	In-store	2023-11-07

```python
# Check data info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Transaction ID    10000 non-null  object
 1   Item              9667 non-null   object
 2   Quantity          9862 non-null   object
 3   Price Per Unit    9821 non-null   object
 4   Total Spent       9827 non-null   object
 5   Payment Method    7421 non-null   object
 6   Location          6735 non-null   object
 7   Transaction Date  9841 non-null   object
dtypes: object(8)
memory usage: 625.1+ KB
```

All attributes in this dataset are of object type.

Data Preparation/ Cleaning

Before the dataset is used for data analysis, data preprocessing is required to ensure the data is accurate, consistent, and suitable for analysis. Data preprocessing in this project is divided into five phases: data transformation and data replacement; checking, replacing, and dropping missing values; correcting inconsistent data; checking for duplicate data; and feature engineering.

1. **Data Type Conversion and Invalid Data Replacement**

   During this step, data must be examined and converted to the appropriate data type to ensure compatibility for analysis. Attributes including Quantity, Price per Unit, and Total Spent were transformed into numeric data types for easier calculations. Furthermore, columns with restricted values, such as Payment method, Location, and item, are converted to categorical data types. Additionally, the transaction date field will be converted to a datetime type to enhance time-series analysis.

```python
# Change data type

df['Item'] = df['Item'].astype('category')

df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce').astype('Int64')

df['Price Per Unit'] = pd.to_numeric(df['Price Per Unit'], errors='coerce')
df['Total Spent'] = pd.to_numeric(df['Total Spent'], errors='coerce')
df['Payment Method'] = df['Payment Method'].astype('category')
df['Location'] = df['Location'].astype('category')
df['Transaction Date'] = pd.to_datetime(df['Transaction Date'], dayfirst=True, format='mixed',errors='coerce')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Transaction ID    10000 non-null  object
 1   Item              9031 non-null   category
 2   Quantity          9521 non-null   Int64
 3   Price Per Unit    9467 non-null   float64
 4   Total Spent       9498 non-null   float64
 5   Payment Method    6822 non-null   category
 6   Location          6039 non-null   category
 7   Transaction Date  9540 non-null   datetime64[ns]
dtypes: Int64(1), category(3), datetime64[ns](1), float64(2), object(1)
memory usage: 430.4+ KB
None
```

Additionally, invalid values in the dataset were identified and replaced. Invalid values, such as “Error” and “Unknown,” will be set to null for subsequent data cleaning.

```python
# replace value "ERROR" AND "UNKNOWN" With null value for better data cleaning
df = df.replace(to_replace=["ERROR", "UNKNOWN"], value= pd.NA)
```

2. <u>Handling Missing Values</u>

After transforming the data type and replacing invalid values, the database can identify missing values. Missing values are caused by data input errors or missing reporting. To identify and address missing data, df.isnull().sum() counts the total number of missing values for each attribute.

```python
# Check missing value
df.isnull().sum()
```

```
Transaction ID         0
Item                 969
Quantity             479
Price Per Unit       533
Total Spent          502
Payment Method      3178
Location            3961
Transaction Date     460
```

The result found that there are 969 missing values in the Item column, 479 records in the Quantity column, 533 instances in the Price Per Unit column, 502 records in the Total Spent column, 3178 missing values in the Payment Method column, 3961 missing values in the Location column, and 460 records in the Transaction Date column.

In the first phase of data cleaning, mode imputation was employed to address missing values in categorical data. Categorical features like Item, Payment method, and Location use this approach, as the mode indicates the most common value in the dataset, thereby ensuring data consistency.

```python
# Fill/replace missing value
# Replace missing value by mode for each category column
df['Item'].fillna(df['Item'].mode()[0], inplace=True)
df['Payment Method'].fillna(df['Payment Method'].mode()[0], inplace=True)
df['Location'].fillna(df['Location'].mode()[0], inplace=True)
```

To ensure data accuracy, the Price Per Unit column was cleaned by assigning the prices based on the menu given for each item. For example, Coffee was assigned a price of 2, Tea was assigned 1.5, Sandwich was assigned 4, Salad was assigned 5, Cake was assigned 3, Cookie was assigned 1, Smoothie was assigned 4, and Juice was assigned 3.

```python
# fill in the price per unit based on the menu given
df.loc[df['Item'] == 'Coffee', "Price Per Unit" ] = 2
df.loc[df['Item'] == 'Tea', "Price Per Unit" ] = 1.5
df.loc[df['Item'] == 'Sandwich', "Price Per Unit" ] = 4
df.loc[df['Item'] == 'Salad', "Price Per Unit" ] = 5
df.loc[df['Item'] == 'Cake', "Price Per Unit" ] = 3
df.loc[df['Item'] == 'Cookie', "Price Per Unit" ] = 1
df.loc[df['Item'] == 'Smoothie', "Price Per Unit" ] = 4
df.loc[df['Item'] == 'Juice', "Price Per Unit" ] = 3
```

Next, missing values in the Quantity column were handled using median imputation, as the column is an integer variable and the median provides a more reliable estimate that lessens the impact of extreme values; the median was selected rather than the mean. Then, missing values in the Total Spent column were calculated using the formula Quantity multiplied by Price Per Unit, since total spending should logically equal the number of items purchased multiplied by the price per item.

```python
# Check mean, median for quantity column and choose the suitable one to replace the missing value
Qtymean = df['Quantity'].mean()
Qtymedian = df['Quantity'].median()

print(Qtymean)
print(Qtymedian)
```

```
3.0244488977955912
3.0
```

```python
# replace missing value in quantity column with median due to quantity column is int type data
df['Quantity'] = df['Quantity'].fillna(Qtymedian)

# fill na value again for the total spent column | (Quantity*Price Per Unit)
df['Total Spent'] = df['Total Spent'].fillna(df['Quantity'] * df['Price Per Unit'])

df.isna().sum()
```

Lastly, the missing value in the Transaction Date column will be handled by dropping the row, as it accounts for only 4.82% of the total records (below 5%) and does not affect the data, thereby preserving accuracy.

```python
# check total missing value for transaction date by total data
naDate = (df['Transaction Date'].isna().sum()/df['Transaction Date'].count()*100)
print(naDate)
```

```
4.821802935010482
```

3. **<u>Data Consistency Validation</u>**

To ensure the Total Spent amount matched the value computed from Quantity × Price Per Unit, a data consistency check was performed. The Total Spent was recalculated using the same formula for all records with conflicting values. This phase increases the reliability of subsequent analysis and ensures that the dataset maintains the logical relationships among related attributes.

```
# check if the data is correct (Quantity * Price per unit  = total Spent)
df[df['Total Spent'] != (df['Quantity'] * df['Price Per Unit'])]

# correct the total spent with replace the true total spent
df.loc[df['Total Spent'] != (df['Quantity'] * df['Price Per Unit']), 'Total Spent'] = df['Quantity'] * df['Price Per Unit']

# recheck again the incorrect total spent
df[df['Total Spent'] != (df['Quantity'] * df['Price Per Unit'])]
```

Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date

4. **Duplicate Record Detection**

After handling missing values, duplicate data will be checked. The `df.duplicated().sum()` function was used to identify the total number of rows with identical values across all columns.

```
# check duplicate data
print(df.duplicated().sum())
```

```
0
```

The result above displayed that there is no duplicate data in this dataset.

5. **Feature engineering**

Feature engineering was performed to create additional useful features from the existing data. The Transaction Date field was used to extract the day, month, year, and week number after converting it to a datetime format. These additional features enable the analyser to examine transaction data more deeply over time.

```
# Create new column for year,month, day and weekday
df["Day"] = df["Transaction Date"].dt.day
df["Month"] = df["Transaction Date"].dt.month_name()
df["Year"] = df["Transaction Date"].dt.year
df["Weekday"] = df["Transaction Date"].dt.day_name()

df
```

	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date	Day	Month	Year	Weekday
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08	8	September	2023	Friday
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16	16	May	2023	Tuesday
2	TXN_4271903	Cookie	4	1.0	4.0	Credit Card	In-store	2023-07-19	19	July	2023	Wednesday
3	TXN_7034554	Salad	2	5.0	10.0	Digital Wallet	Takeaway	2023-04-27	27	April	2023	Thursday
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11	11	June	2023	Sunday
...	...	...	...	...	...	...	...	...	...	...	...	...
9995	TXN_7672686	Coffee	2	2.0	4.0	Digital Wallet	Takeaway	2023-08-30	30	August	2023	Wednesday
9996	TXN_9659401	Juice	3	3.0	9.0	Digital Wallet	Takeaway	2023-06-02	2	June	2023	Friday
9997	TXN_5255387	Coffee	4	2.0	8.0	Digital Wallet	Takeaway	2023-03-02	2	March	2023	Thursday
9998	TXN_7695629	Cookie	3	1.0	3.0	Digital Wallet	Takeaway	2023-12-02	2	December	2023	Saturday
9999	TXN_6170729	Sandwich	3	4.0	12.0	Cash	In-store	2023-11-07	7	November	2023	Tuesday

9540 rows × 12 columns

After all data preprocessing steps are done, we can find that 9540 records remain and 12 attributes in this dataset.

Exploratory Data Analysis (EDA)

After the data preprocessing phase, the data is considered clean and ready for analysis. In the data analysis phase, MySQL is used to generate insights and answer business questions.

1. **Total Revenue, Total items sold, and Total Transactions** - What is the total revenue, total items sold, and total transactions generated by the cafe in 2023?

```
# Ques 1 : What is the total revenue, total item sold and toatl trasaction generated by the café in 2023
SELECT
    SUM(`Total Spent`) AS Total_Revenue, SUM(`Quantity`) AS Total_Item_Sold, COUNT(`Transaction ID`) AS Total_Transaction
FROM cafe_sale.cafe;
```

sult Grid | Filter Rows: | Exp

Total_Revenue	Total_Item_Sold	Total_Transaction
85316.5	28817	9540

Based on the results presented, the total revenue was $85316.50, the total number of items sold was 28817, and 9540 customer transactions were made in 2023.

2. **Revenue by Month** - How much revenue is generated per month?

```sql
# Ques 2: How much revenue is generated per month
SELECT
    Month, SUM(`Total Spent`) AS Monthly_Revenue
FROM cafe_sale.cafe
GROUP BY Month
order by Monthly_Revenue DESC ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell C

Month	Monthly_Revenue
October	7403
June	7360
January	7344
March	7237.5
April	7223.5
December	7171
August	7119
November	7002
May	6979
September	6958
July	6897
February	6622.5

According to the results, October had the highest revenue at this cafe in 2023 ($7403), followed by June ($7360) and January ($7344). In February, revenue is recorded as the lowest at $6622.5.

3. **Top 3 products by revenue** - Which Items generate the highest total revenue?

```sql
# Ques 3: Which items generate the highest total revenue?
SELECT
    Item, SUM(`Total Spent`) AS Total_Revenue
FROM cafe_sale.cafe
GROUP BY Item
ORDER BY Total_Revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Item	Total_Revenue
Juice	18357
Salad	16605
Sandwich	13116
Smoothie	12800
Cake	9843
Coffee	6844
Tea	4669.5
Cookie	3082

Juice, Salad, and Sandwich are the three items that result in the highest revenue. Juice is the highest at $18,357, followed by Salad at $16,605, and Sandwich at $13,116. The item that generated the lowest revenue was the cookie at $3082.

4. **Top 3 Products by Quantity** - Popular Item

```sql
#Ques 4: Which items are sold the most by quantity?
SELECT
    Item, SUM(Quantity) AS Total_Quantity
FROM cafe_sale.cafe
GROUP BY Item
ORDER BY Total_Quantity DESC;
```

sult Grid | Filter Rows: | Export: | Wrap Cell Cor

Item	Total_Quantity
Juice	6119
Coffee	3422
Salad	3321
Cake	3281
Sandwich	3279
Smoothie	3200
Tea	3113
Cookie	3082

Juice at the cafe in 2023 had the highest sales, with 6119 sold, followed by coffee at 3422 and salad at 3321. The lowest number of items sold was cookies at 3082.

5. **Payment Method Comparison**

```sql
# Ques 5: Which payment method is used most frequently?
SELECT
    `Payment Method`, Count(`Payment Method`) AS Count_Payment
FROM cafe_sale.cafe
GROUP BY `Payment Method`
ORDER BY Count_Payment DESC;
```

sult Grid | Filter Rows: | Export: | Wrap Cell Content:

Payment Method	Count_Payment
Digital Wallet	5212
Credit Card	2170
Cash	2158

In 2023, the majority of customers used the digital wallet to make payments. A total of 5212 payments are made using a digital wallet. Then, 2170 payments were made by credit card, while 2158 were made in cash.

6. Revenue by Payment Method

```sql
# Ques 6: How does revenue differ by payment method?
SELECT
    `Payment Method`, SUM(`Total Spent`) AS Total_Revenue
FROM cafe_sale.cafe
GROUP BY `Payment Method`
ORDER BY Total_Revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Payment Method	Total_Revenue
Digital Wallet	46191.5
Cash	19565.5
Credit Card	19559.5

Payment via digital wallet recorded the highest revenue at $46191.5, while cash payment recorded $19565.5, and credit card recorded the lowest revenue at $19559.5.

7. Location Comparision

```sql
# Ques 7: Which location is the customer having the item?
SELECT
    `Location`, Count(`Location`) AS Count_Location
FROM cafe_sale.cafe
GROUP BY `Location`
ORDER BY Count_Location DESC;
```

sult Grid | Filter Rows: | Export: | Wrap Cell Content:

Location	Count_Location
Takeaway	6668
In-store	2872

The customer who takes away the item (6668) is greater than in-store (2872).

8. Revenue by Location

```
# Ques 8: Do In-store or Takeaway orders generate more revenue?
SELECT
    Location, SUM(`Total Spent`) AS Total_Revenue
FROM cafe_sale.cafe
GROUP BY Location
ORDER BY Total_Revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Location	Total_Revenue
Takeaway	59193.5
In-store	26123

In 2023, takeaway customers accounted for $59,193.50 of the café's revenue, while in-store customers accounted for $26,123.

9. Revenue by Weekday

Limit to 1000 rows

```
# Ques 9: Which day of the week generates the highest revenue?
SELECT
    Weekday, SUM(`Total Spent`) AS Total_Revenue
FROM cafe_sale.cafe
GROUP BY Weekday
ORDER BY Total_Revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

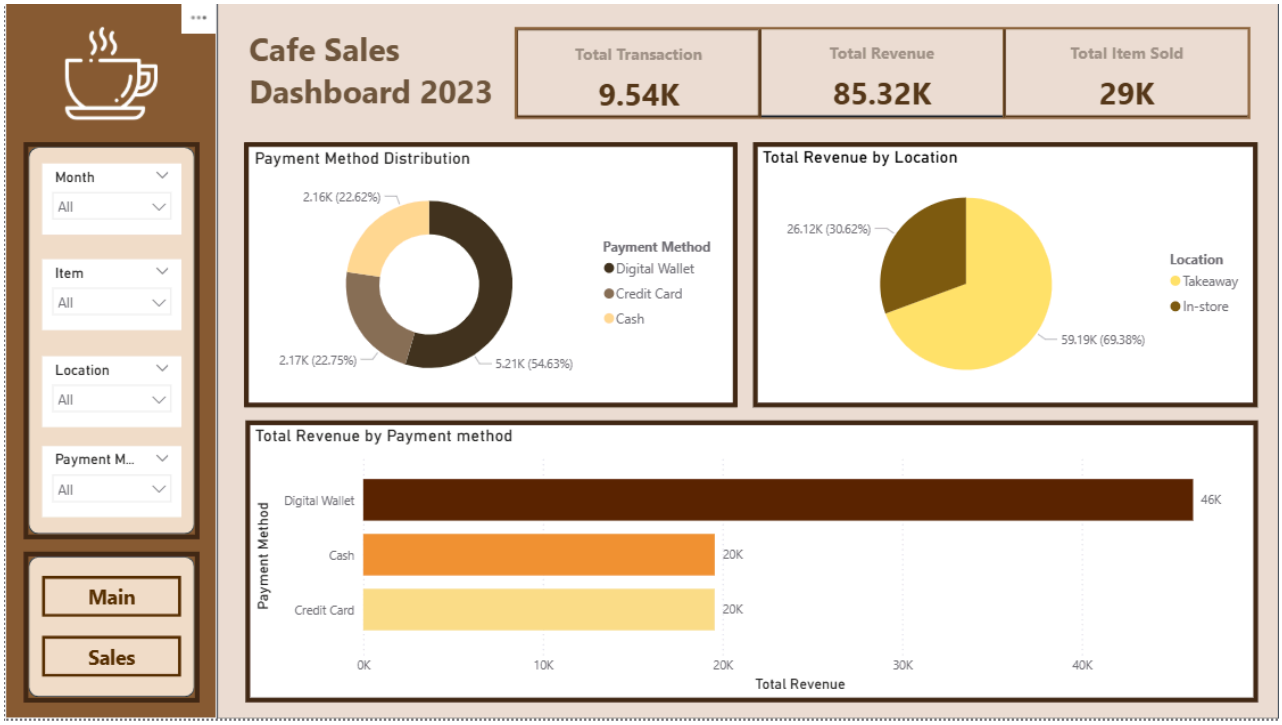
Weekday	Total_Revenue
Friday	12492
Thursday	12412.5
Sunday	12340.5
Monday	12142.5
Saturday	12127.5
Tuesday	12113
Wednesday	11688.5

Friday is a peak day for sales with $12492, followed by Thursday at $12412.5 and Sunday at $12340.5. Wednesday's revenue scored the lowest number at $11688.5.

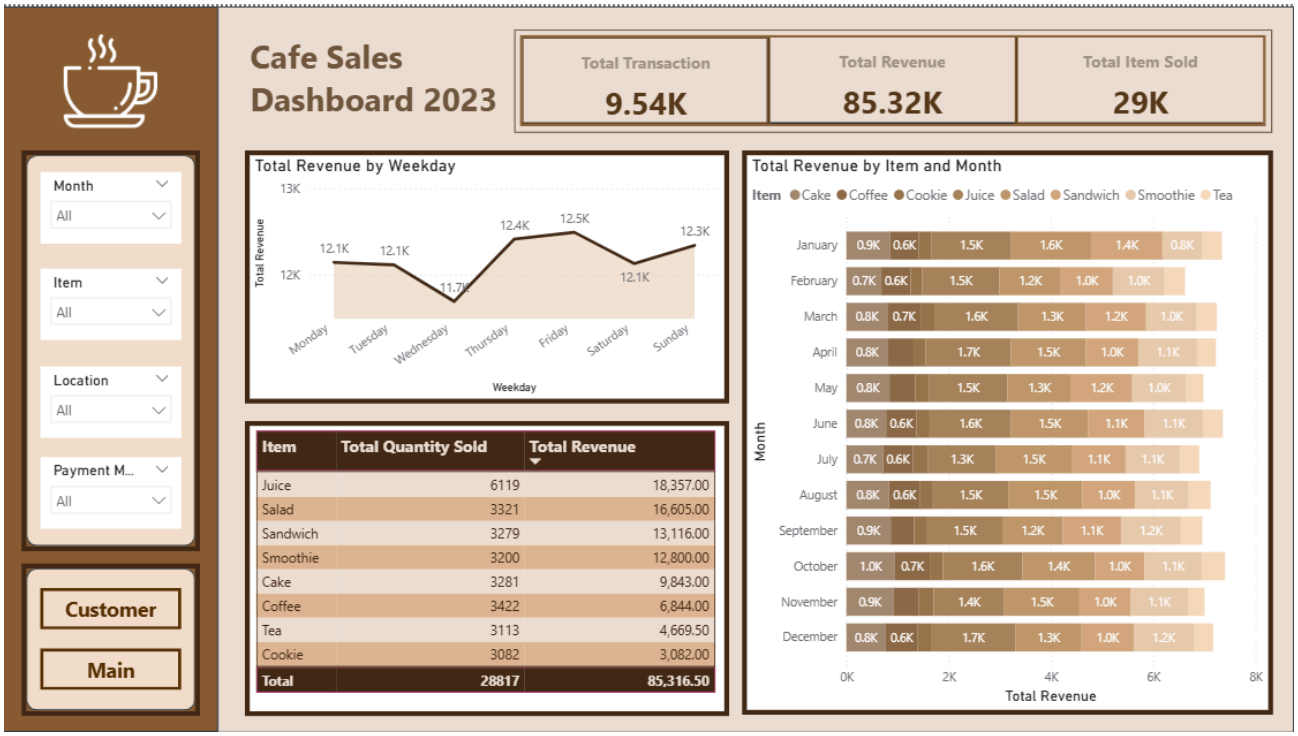
Data Visualization in Power BI

Finally, the results generated by MySQL will be visualised in the Power BI report, where users can view the pattern and other details. This report is divided into three parts: the main page, the customer behaviour page, and the sales details page. All pages cover Total Transaction, Total revenue, and Total Item Sold, and the user can view a specific record using the slicers to select classes (Month, Item, Location, Payment Method). The main page displayed the total revenue by month, the total number of items sold, and the total revenue by Item. However, the customer behaviour page displayed the distribution of payment methods and total revenue by location and payment method. The sales details page illustrated total revenue by weekday, revenue by item and month, and a table detailing sales.

Main Page



Customer Behavior Page



Item	Total Quantity Sold	Total Revenue
Juice	6119	18,357.00
Salad	3321	16,605.00
Sandwich	3279	13,116.00
Smoothie	3200	12,800.00
Cake	3281	9,843.00
Coffee	3422	6,844.00
Tea	3113	4,669.50
Cookie	3082	3,082.00
Total	**28817**	**85,316.50**

Sales Details Page

Business Recommendations

1. Festival Promotions - February had the lowest revenue in 2023. The cafe owner can organise an activity to celebrate the February festival, such as Valentine's Day. The cafe owner can focus sales on coffee, cookies, tea, and cake to increase revenue and item sales.
2. Bundle sets - The highest-selling and revenue-generating products, such as juice and salad, can be bundled under a special set name (e.g., Healthy daily set) at a discounted price to increase average transaction value.
3. Optimize payment method discount - Cafe owners can collaborate with a digital wallet provider, such as Touch n Go or GrabPay, to offer a discount (5% - 10%) to improve transaction speed.
4. Improve the revenue of low-selling items - For example, a cafe owner can buy coffee or tea and get 20% off a cookie to boost cookie visibility and sales.
5. Weekly promotion - Cafe owners can choose to have the promotion every Wednesday, named “Happy Wednesday or Wed Cafe Day”, with 10% off smoothies and sandwiches to attract customers during slower periods.